

Formal Languages and Compilers

(Linguaggi Formali e Compilatori)

prof. Luca Breveglieri

(prof. A. Morzenti)

Written exam - 5 March 2014 - Part I: Theory

WITH SOLUTIONS - FOR TEACHING PURPOSES HERE THE SOLUTIONS ARE WIDELY

COMMENTED - THE CANDIDATE SHOULD CORRECTLY ANSWER AND REASONABLY EXPLAIN WHY

WARNING - THE SYNTAX ANALYSIS EXERCISES MUST BE SOLVED BY THE ELR / ELL THEORY

NAME:

MATRICOLA:

SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam consists of two parts:
 - I (80%) Theory:
 1. regular expressions and finite automata
 2. free grammars and pushdown automata
 3. syntax analysis and parsing
 4. translation and semantic analysis
 - II (20%) Practice on Flex and Bison
- To pass the exam, the candidate must succeed in both parts (I and II), in one call or more calls separately, but within one year.
- To pass part I (theory) one must answer the mandatory (not optional) questions. Notice the full grade is achieved by answering the optional questions.
- The exam is open book (texts and personal notes are admitted).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets nor replace the existing ones.
- Time: Part I (theory): 2h.15m - Part II (practice): 60m

1 Regular Expressions and Finite Automata 20%

1. Consider the right-linear grammar G below (axiom S), over the terminal alphabet $\{a, b, c\}$:

$$G \left\{ \begin{array}{l} S \rightarrow a A \\ A \rightarrow a B \mid \varepsilon \\ B \rightarrow c S \mid b A \end{array} \right.$$

Answer the following questions:

- (a) By means of the method of the linear language equations, find a regular expression R equivalent to grammar G .
 - (b) By means of the Berry-Sethi method, applied to the regular expression R found before, build a deterministic finite automaton A that recognizes the language $L(G)$ of grammar G .
 - (c) (Optional) Build a deterministic finite automaton A' that recognizes the language $\neg L(G)$, i.e., the complement language of $L(G)$.
-

Solution

- (a) Here is the solution of the system of linear equations of grammar G , with unknown variables L_S , L_A and L_B :

$$\begin{cases} L_S = \{ a \} L_A \\ L_A = \{ a \} L_B \mid \{ \varepsilon \} \\ L_B = \{ c \} L_S \mid \{ b \} L_A \end{cases}$$

Replace the first equation into the third one:

$$\begin{cases} L_S = \{ a \} L_A \\ L_A = \{ a \} L_B \mid \{ \varepsilon \} \\ L_B = \{ c \} \{ a \} L_A \mid \{ b \} L_A \\ \quad = \{ ca \} L_A \mid \{ b \} L_A \\ \quad = (\{ ca \} \mid \{ b \}) L_A \end{cases}$$

Then replace the third equation into the second one:

$$\begin{cases} L_S = \{ a \} L_A \\ L_A = \{ a \} (\{ ca \} \mid \{ b \}) L_A \mid \{ \varepsilon \} \\ L_B = (\{ ca \} \mid \{ b \}) L_A \end{cases}$$

The second equation becomes recursive, and can be shortly written this way:

$$L_A = a (ca \mid b) L_A \mid \varepsilon$$

Now use the Arden identity for solving it and getting the unknown L_A :

$$L_A = (a (ca \mid b))^*$$

Finally replace into the first equation and get the final language L_S :

$$L_S = a (a (ca \mid b))^* = R$$

So now we have a regular expression R for the language $L(G) = L(S)$. One could also quickly obtain the unknown L_B , but it is not required.

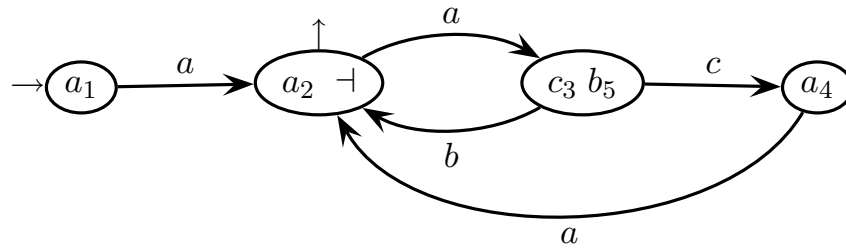
(b) Apply Berry-Sethi to the regular expression R . First number it:

$$R_{\#} = a_1 \left(a_2 \left(c_3 a_4 \mid b_5 \right) \right)^*$$

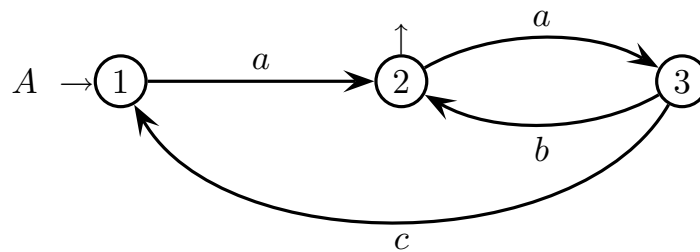
Then compute the initials and the followers:

initials	a_1
generators	followers
a_1	$a_2 \dashv$
a_2	$c_3 b_5$
c_3	a_4
a_4	$a_2 \dashv$
b_5	$a_2 \dashv$

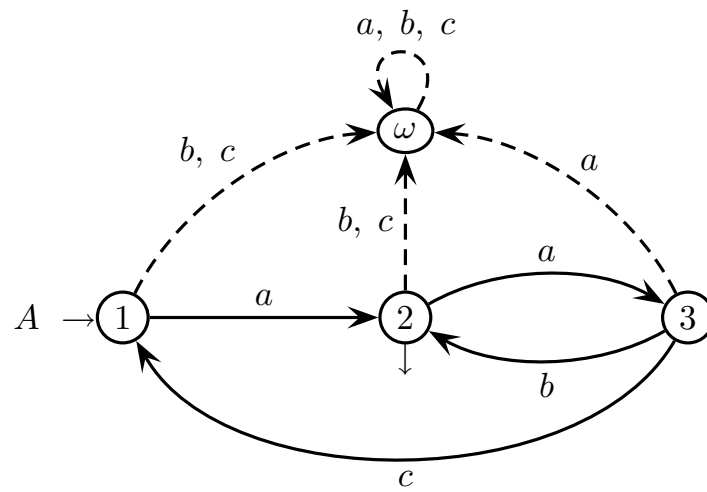
Now draw a deterministic automaton:



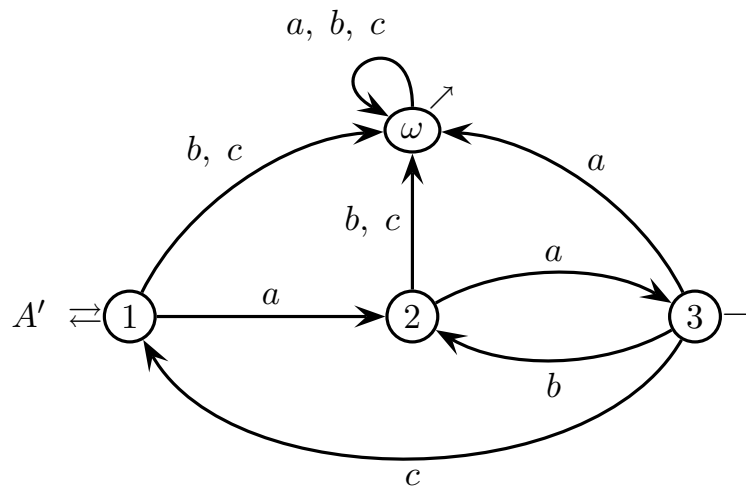
It is clear that this automaton is not minimal, as the non-final states a_1 and a_4 are undistinguishable. There are not any other undistinguishable state pairs. Finally the minimal deterministic automaton A is the following one:



(c) Apply the complement construction. First introduce the error state ω :



Then switch the final and non-final states, and eventually get the complement deterministic automaton A' :



It is clear that the deterministic automaton A' is minimal as well.

2 Free Grammars and Pushdown Automata 20%

1. Consider the two following grammars G_1 (with axiom S):

$$G_1 \left\{ \begin{array}{l} S \rightarrow a S b S \mid a S_1 b S_1 \\ S_1 \rightarrow c S_1 d S_1 \mid \varepsilon \end{array} \right.$$

and G_2 (with axiom S):

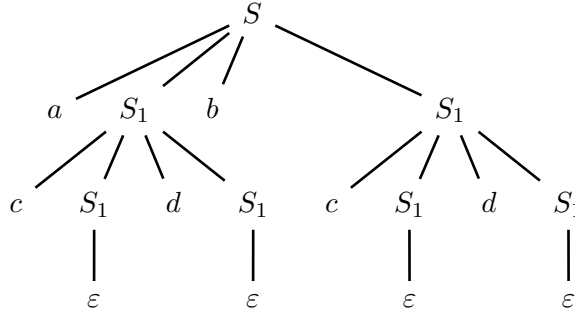
$$G_2 \left\{ \begin{array}{l} S \rightarrow S e S f \mid S_1 e S_1 f \\ S_1 \rightarrow S_1 c S_1 d \mid \varepsilon \end{array} \right.$$

Answer the following questions:

- (a) Draw the syntax tree of string $a c d b c d \in L(G_1)$
 - (b) Write a free grammar G_3 of any type (no matter if it is ambiguous) that generates the concatenation language $L(G_1) \cdot L(G_2)$. Draw a syntax tree of string $a b c d e f$, according to grammar G_3 . Say if grammar G_3 is ambiguous or not, and explain why.
 - (c) (Optional) If the grammar G_3 previously defined is ambiguous, then define a new grammar G_4 that is not ambiguous, and shortly explain why it is not ambiguous.
-

Solution

- (a) Here is the syntax tree of string $a c d b c d \in L(G_1)$:



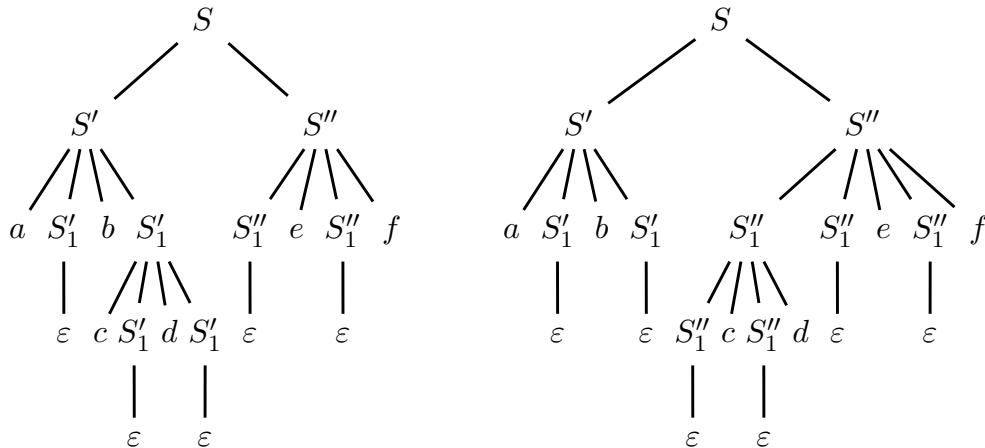
and is the only possible tree for that string since grammar G_1 is not ambiguous, as well as grammar G_2 .

The language of grammar G_1 is basically a restriction of Dyck, such that the matching pairs are ab and cd , but a pair cd may not contain any pair ab . Similarly with the language of G_2 , which has pair ef instead of pair ab .

- (b) It suffices to disjoin the nonterminal alphabets of the two grammars and to concatenate their axioms. Here is grammar G_3 (axiom S):

$$G_3 \left\{ \begin{array}{l} S \rightarrow S' S'' \\ \hline S' \rightarrow a S' b S' \mid a S'_1 b S'_1 \\ S'_1 \rightarrow c S'_1 d S'_1 \mid \varepsilon \\ \hline S'' \rightarrow S'' e S'' f \mid S''_1 e S''_1 f \\ S''_1 \rightarrow S''_1 c S''_1 d \mid \varepsilon \end{array} \right.$$

The grammar G_3 above is ambiguous. For instance string $a b c d e f \in L(G_3)$ has two syntax trees:



This happens with every inner substring of c, d letters, which separates the a, b prefix produced by G_1 from the e, f suffix produced by G_2 .

- $$G_4 \left\{ \begin{array}{l} S \rightarrow S' S'' \\ \hline S' \rightarrow a S' b S' \mid a S'_1 b S'_1 \\ S'_1 \rightarrow c S'_1 d S'_1 \mid \varepsilon \\ \hline S'' \rightarrow e S'' f S'' \mid e S''_1 f \\ S''_1 \rightarrow S''_1 c S''_1 d \mid \varepsilon \end{array} \right.$$

```

graph TD
    S --> S_prime[S']
    S --> S_double_prime[S'']
    S_prime --> a
    S_prime --> S_prime_1[S'_1]
    S_prime --> b
    S_double_prime --> e
    S_double_prime --> S_double_prime_1[S''_1]
    S_double_prime --> f
    S_prime_1 --> epsilon_1[ε]
    S_prime_1 --> c
    S_prime_1 --> S_prime_1_1[S'_1]
    S_prime_1 --> d
    S_double_prime_1 --> epsilon_2[ε]
    S_prime_1_1 --> epsilon_3[ε]
    
```

8

2. One wishes one modeled the syntax of the declaration section of the variables in the C language, with some restrictions. Here are the relevant characteristics to be modeled:

- a variable name is an identifier, symbolized by the terminal `id`
- an integer constant is symbolized by the terminal `num`
- the scalar type are `int`, `char` and `float`, the first one with the optional attribute `unsigned`
- the structured types are the following ones:
 - the record or structure `struct`
 - the array of one or two dimensions, with elements of type scalar or record
- it is also admitted to declare pointers, of any level
- the sign “`*`” for declaring pointers is right-associative
- the sign for declaring arrays (i.e., the square brackets “`[]`”) is left-associative

For anything that is not explicitly specified by the above list, do as it is specified by the standard of the C language.

Here is an example of a few variable declarations:

```
char alpha, beta2, BIG;

unsigned int a, b, cc;

struct {
    float * f1;           -- f1 is a pointer
    char * * omega;       -- omega is a pointer (of level 2)
} rec;

int matrix [10] [20];
```

Answer the following questions:

- Write a grammar, in the extended form (*EBNF*) and not ambiguous, that reasonably models the language fragment described above.
- (Optional) Suppose one wanted to add the possibility of defining new types, by means of the `typedef` construct. Here is an example of a few type declarations:

```
typedef unsigned int UINT;    -- UINT is a new type
typedef char VECT [17];      -- VECT is a new type

typedef struct {
    ...
    ...
} * PREC, SUCC;              -- PREC e SUCC are new types
```

Extend the grammar found at the point (a) (still writing an *EBNF* and not ambiguous grammar). Show only the necessary new rules and / or those that are modified.

Solution

- (a) Here is the required grammar, in the *EBNF* form (axiom `D_SECT`):

$$\left\{ \begin{array}{l} \langle \text{D_SECT} \rangle \rightarrow (\langle \text{DECL} \rangle \text{ ' ; ' })^+ \\ \hline \langle \text{DECL} \rangle \rightarrow \langle \text{SCALAR} \rangle \mid \langle \text{STRUCT} \rangle \mid \langle \text{ARRAY} \rangle \\ \hline \langle \text{SCALAR} \rangle \rightarrow ([\text{unsigned}] \text{ int } \mid \text{char} \mid \text{float}) \langle \text{PTRS} \rangle \\ \langle \text{STRUCT} \rangle \rightarrow \text{struct ' \{ ' } \langle \text{D_SECT} \rangle \text{ ' \} ' } \langle \text{PTRS} \rangle \\ \hline \langle \text{ARRAY} \rangle \rightarrow (\langle \text{SCALAR} \rangle \mid \langle \text{STRUCT} \rangle) [\langle \text{INDEX} \rangle] \text{ ' [' num '] ' } \\ \hline \langle \text{PTRS} \rangle \rightarrow \text{ ' * ' } \langle \text{PTRS} \rangle \mid \langle \text{ID_LIST} \rangle \\ \langle \text{INDEX} \rangle \rightarrow \text{ ' [' num '] ' } \\ \hline \langle \text{ID_LIST} \rangle \rightarrow \text{ id (' , ' id) }^* \end{array} \right.$$

This grammar is not ambiguous. As usual the square brackets mean optionality.

- (b) Here are the modified and new rules for the required grammar with `typedef`, in the *EBNF* form (axiom `D_SECT`):

$$\left\{ \begin{array}{l} \langle \text{DECL} \rangle \rightarrow \langle \text{SCALAR} \rangle \mid \langle \text{STRUCT} \rangle \mid \langle \text{ARRAY} \rangle \mid \langle \text{TYPEDEF} \rangle \\ \langle \text{TYPEDEF} \rangle \rightarrow \text{typedef (} \langle \text{SCALAR} \rangle \mid \langle \text{STRUCT} \rangle \mid \langle \text{ARRAY} \rangle \text{)} \end{array} \right.$$

Rule `DECL` has been modified and rule `TYPEDEF` has been added. The rest of the grammar (not shown) is unchanged. This grammar is not ambiguous either. Notice that a clause `typedef` may occur inside of a record. This is admitted in the C language, but the scope of the clause is local to the record.

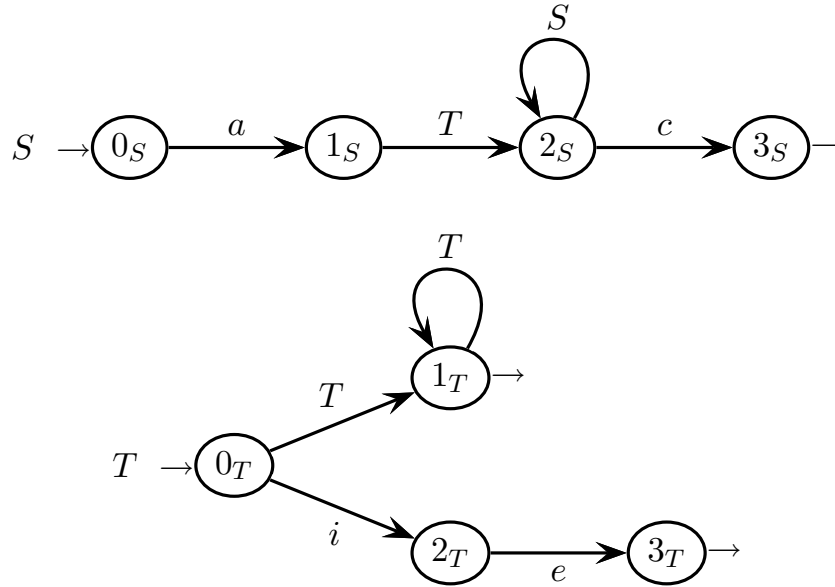
The new grammar could be compacted in this way:

$$\left\{ \begin{array}{l} \langle \text{DECL} \rangle \rightarrow [\text{typedef}] (\langle \text{SCALAR} \rangle \mid \langle \text{STRUCT} \rangle \mid \langle \text{ARRAY} \rangle) \end{array} \right.$$

by dropping the nonterminal `TYPEDEF`.

3 Syntax Analysis and Parsing 20%

1. Consider the grammar G defined by the recursive machines below (axiom S):

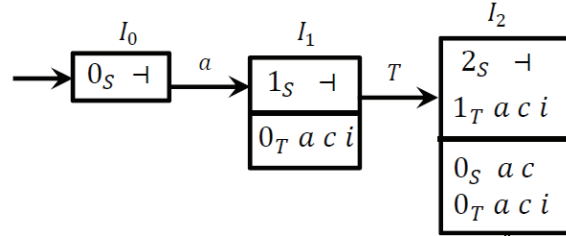


Answer the following questions:

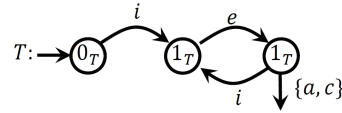
- (a) Build the smallest part of the *ELR* pilot graph of grammar G that allows to show that grammar G is not *ELL*(1). Make sure that the m-stats of the drawn pilot part contain all the necessary details, including the look-ahead sets.
- (b) On the machines that represent grammar G , write the guide sets that prove that grammar G is not *ELL*(1).
- (c) Modify grammar G and obtain a variant grammar G' that is *ELL*(1), of course without changing the generated language. On the machines of the new grammar G' , show that the guide sets satisfy the *ELL*(1) condition.
- (d) (Optional) Continue to build the *ELR* pilot automaton of grammar G (started at point (a)), as much as it suffices to show that grammar G is not *ELR*(1).

Solution

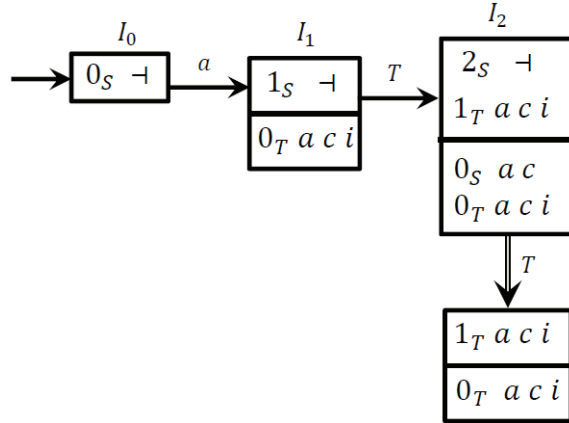
- (a) Here is the required part of the *ELL* pilot graph, which does not satisfy the *STP* property, for transition $I_1 \xrightarrow{T} I_2$ is multiple (more precisely double).



- (b) The relevant guide sets are those on the two arcs outgoing state 0_T , which are both equal to $\{i\}$, and on the two arcs outgoing state 1_T , that is $\{i\}$ and $\{i, a, c\}$, which are not disjoint.
- (c) The problem originates from the left recursion of nonterminal T , but since it holds $L(T) = (ie)^+$, a regular language, then the simple machine below satisfies the *ELL*(1) condition.



- (d) The part of pilot graph below shows that the grammar is not *ELL*(1) because of a conflicting convergent arc (the double-lined one).



In fact the two candidates $\langle 1_T, \{a, c\} \rangle$ and $\langle 0_T, \{a, c, i\} \rangle$ of m-state I_2 , which both converge to state 1_T by shifting on symbol T , share the look-ahead symbols a and c . Of course there may be also other conflicts, of different type.

4 Translation and Semantic Analysis 20%

1. Consider the arithmetic expressions with the addition operator “+”, but without parentheses, and with variables and constants symbolized by “ v ”. Such expressions must be translated into a target language that, besides the standard addition operator “+”, also has a fast addition function (called *fadd*), denoted as “*fadd*(*arg*₁, *arg*₂)”, with two arguments. The translation tries to substitute the function *fadd* to the standard addition operator “+”, which is less efficient. But the translation can be done in different ways, more or less optimized, as it is explained below.

Answer the following questions:

- (a) One wishes one translated an addition expression by using the function *fadd* as much as possible, yet without using recursive function calls (nested functions) since the target language does not support them. Therefore the summands are pairwise translated with *fadd* starting from the left, and the last summand may be left alone with the operator “+”. Here are a few examples:

<i>source</i>	<i>translation</i>
v	v
$v + v$	$fadd(v, v)$
$v + v + v$	$fadd(v, v) + v$
$v + v + v + v$	$fadd(v, v) + fadd(v, v)$

Say if this translation is regular or syntactic, and consequently write the regular translation expression or the syntactic translation scheme that defines it.

- (b) One wishes one translated an addition expression by using only the function *fadd*, and by resorting to recursive function calls (nested functions) since the target language supports them. Addition must be considered ***right-associative***. Here are a few examples:

<i>source</i>	<i>translation</i>
v	v
$v + v$	$fadd(v, v)$
$v + v + v$	$fadd(v, fadd(v, v))$

Say if this translation is regular or syntactic, and consequently write the regular translation expression or the syntactic translation scheme that defines it.

- (c) (Optional) Discuss if the translations considered at points (a) and (b), are deterministic or not. Examine the options available to design deterministic automata (finite state or pushdown depending on the case) that compute such translations.

Solution

- (a) The transduction is regular. Here is a regular translation expression for it:

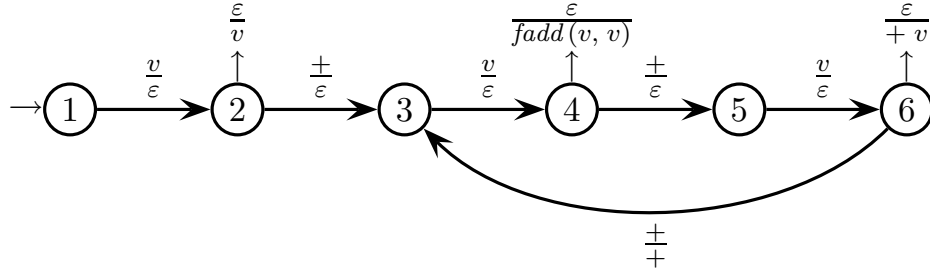
$$\frac{v}{v} \left| \frac{v \text{ ' + ' } v}{fadd('v \text{ ' , ' } v \text{ '})} \left(\frac{\text{ ' + ' } v \text{ ' + ' } v}{\text{ ' + ' } fadd('v \text{ ' , ' } v \text{ '})} \right)^* \left[\frac{\text{ ' + ' } v}{\text{ ' + ' } v} \right] \right.$$

which is quite natural and does not deserve any comment.

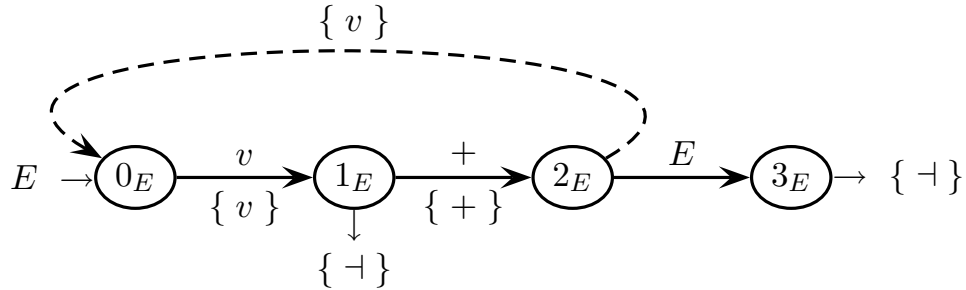
- (b) The transduction is not regular, because the destination language consists of nested structures of arbitrary depth. However it is syntactic, for the same reason. Here is a syntactic scheme for it (axiom E), with a right recursive source grammar (the destination one has a self-embedded recursion):

$$\left\{ \begin{array}{ll} E \rightarrow v \text{ ' + ' } E & E \rightarrow fadd('v \text{ ' , ' } E \text{ '}) \\ E \rightarrow v & E \rightarrow v \end{array} \right.$$

- (c) Translation (a) is deterministic with a finite state translator, which is easy to draw as a subsequential machine (that is, a machine with output also on the exit darts of the final states). Here it is:

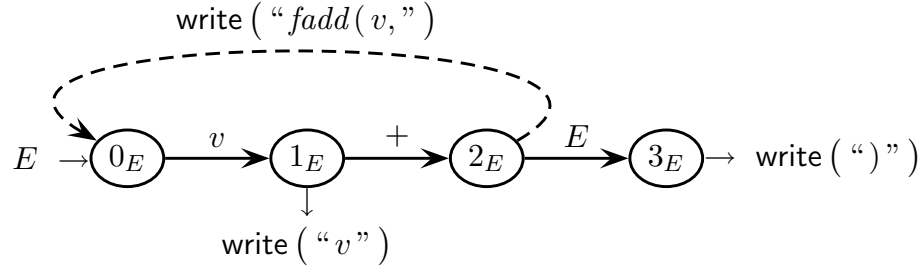


Translation (b) is deterministic: the source grammar is $LL(2)$, since the first and second rule have disjoint guide sets $\{v +\}$ and $\{v \text{ ' + ' }\}$ with $k = 2$, respectively. When represented as a machine, the grammar even turns out to be $ELL(1)$, because the machine is a deterministic FA and unifies the prefix v of the two rules. Here is the source ELL pilot, with call arcs and guide sets:



At the only bifurcation point on state 1_E , the guide sets are disjoint, so the parser is deterministic.

Here is the *ELL* pilot completed with the write actions on the arcs (for simplicity the guide sets are removed):



It is easy to write a recursive descent translator for it. Here it is (as usual function **next** reads one input character or the terminator):

procedure <i>E</i>		program <i>TRANSDUCER</i>
if <i>cc</i> = <i>v</i> then	-- 0 _E	next
next		call <i>E</i>
if <i>cc</i> = + then	-- 1 _E	if <i>cc</i> = + then
next		translated
if <i>cc</i> = <i>v</i> then	-- 2 _E	else
write ("fadd(<i>v</i> , ")		error
call <i>E</i>		end if
if <i>cc</i> = + then	-- 3 _E	end program
write (") ")		
return		
else error		
end if		
else error	-- 2 _E	
end if		
else if <i>cc</i> = + then	-- 1 _E	
write ("v")		
return		
else error		
end if		
else error	-- 0 _E	
end if		
end procedure		

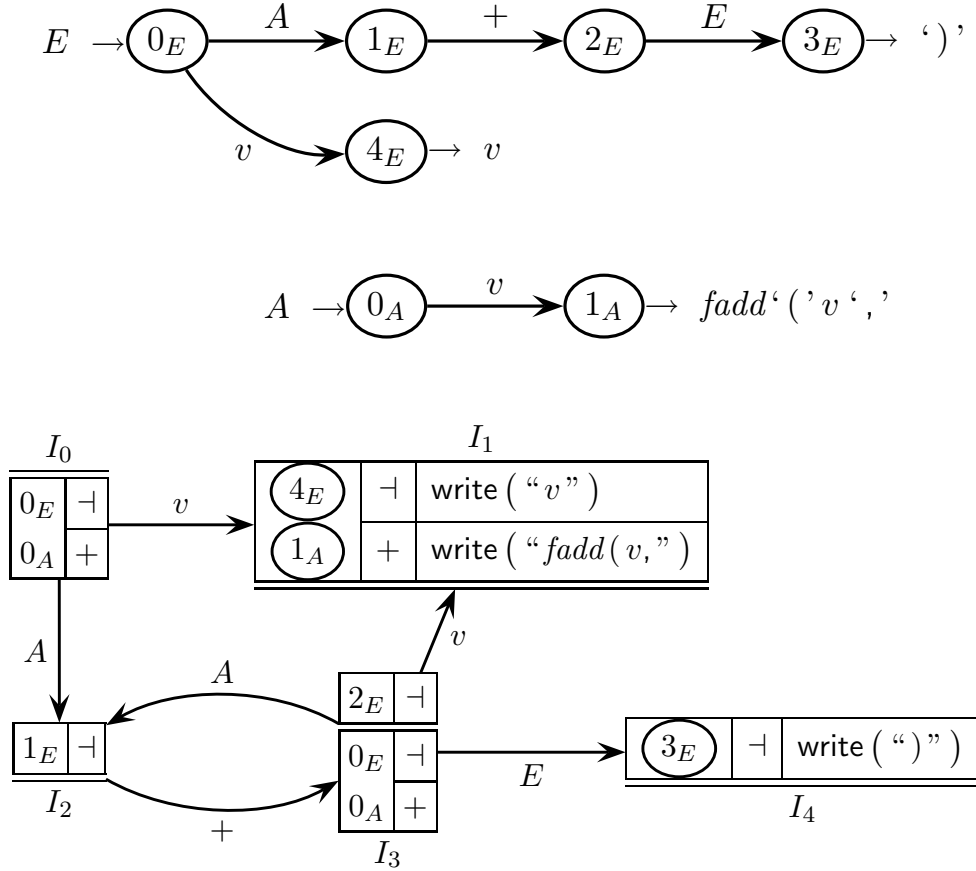
The syntactic procedure *E* could be largely optimized, of course.

To complete the picture, since the source grammar is *ELL*(1), it is also *ELR*(1). Yet the syntactic scheme does not have a postfix form. However it can be easily put into

postfix form (with all output terminals only at the rule end), this way (axiom E):

$$\left\{ \begin{array}{ll} E \rightarrow A \text{ ' + ' } E & E \rightarrow A E \text{ ') ' } \\ E \rightarrow v & E \rightarrow v \\ A \rightarrow v & A \rightarrow fadd \text{ ' (' } v \text{ ' , ' } \end{array} \right.$$

by introducing an auxiliary nonterminal A . Now the destination grammar only has terminals at the end of its rules. But the postfix scheme might no longer have the *ELR* property. To check, it is necessary to draw the new (source) machine net (with output terminals on its exit darts) and then the new pilot graph. Here they are:



The pilot graph satisfies the *ELR* condition: there are no m-states with both shift and reduction; the two reductions in the m-state I_1 have disjoint look-ahead; and there is a multiple transition indeed, but it is not convergent, hence it cannot be conflicting. So the bottom-up translator is deterministic. The pilot also shows the write actions, associated with the three reduction states. By the way, notice that this postfix scheme is not *ELL*, because the new pilot graph does not have the *STP*.

2. Consider the following syntactic support (axiom S):

$$\left\{ \begin{array}{l} S \rightarrow E \\ E \rightarrow E \text{ ' + ' } E \\ E \rightarrow \text{ ' (' } E \text{ ') ' } \\ E \rightarrow n \end{array} \right.$$

This syntax generates strings that represent simple additive expressions with parentheses, where the terminal n symbolizes the elementary summands.

Answer the following questions:

- (a) Write an attribute grammar that computes, in a suitable attribute of the root, the maximum number of summands (a summand may be an elementary one or a subexpression between parentheses) that are present at the outer level or that are contained in a subexpression at an inner level. For instance for the following expression:

$$\underbrace{n + \underbrace{(n + n + n)}_3}_2$$

the maximum is 3; for $n + (n + n + n) + n + n$ it is 4; and for $(n + n) + (n + (n + n + n + n + n) + n + n) + n + n$ it is 5. A solution can be written with only two attributes. See the tables on the next pages.

- (b) Say if the attribute grammar defined at the previous point is one-sweep or not, say if it is L or not, and explain why.
- (c) (Optional) Write a new attribute grammar variant of the previous one (but without changing the syntax support) that, in addition to the maximum number of summands as specified above, also computes the parenthetic nesting level where it is located the subexpression with the maximum number of summands. Review the sample cases listed before. For the following expression:

$$\underbrace{n + \underbrace{(n + n + n)}_{\text{num } 3 \quad \text{liv } 1}}_{\text{num } 2 \quad \text{liv } 0}$$

the maximum is 3 and the level is 1; for $n + (n + n + n) + n + n$ the maximum is 4 and the level is 0; and for $(n + n) + (n + (n + n + n + n + n) + n + n) + n + n$ the maximum is 5 and the level is 2.

If there are two or more (sub)expressions that have the maximum number of summands, but that are located at different nesting levels, the nesting level to be computed can be selected at choice among the levels of any such (sub)expressions. For instance, for the expression $n + (n + n + n) + n$ the maximum is 3, and the level may be 0 or 1.

A solution can be obtained by adding some attributes and the related semantic functions to the solution of point (a). See the tables in the next pages.

Say if the variant attribute grammar is one-sweep or not, say if it is L or not, and explain why.

attributes to be used for the grammar - question (a) - already defined

<i>type</i>	<i>name</i>	<i>domain</i>	<i>nonterm.</i>	<i>meaning</i>
left	<i>a</i>	integer	<i>E</i>	number of summands at the same nesting level in an expression
left	<i>amax</i>	integer	<i>E, S</i>	maximum number of summands at the same nesting level in a subexpression

attributes to be added for question (c)

<i>type</i>	<i>name</i>	<i>domain</i>	<i>nonterm.</i>	<i>meaning</i>

$$1: S_0 \rightarrow E_1$$

$$2: E_0 \rightarrow E_1 \text{ ' + ' } E_2$$

$$3: E_0 \rightarrow \text{ ' (' } E_1 \text{ ') ' }$$

$$4: E_0 \rightarrow n$$

$$1: S_0 \rightarrow E_1$$

$$2: E_0 \rightarrow E_1 \text{ ' + ' } E_2$$

$$3: E_0 \rightarrow \text{ ' (' } E_1 \text{ ') ' }$$

$$4: E_0 \rightarrow n$$

Solution

- (a) Here is the basic attribute grammar (for brevity we use the predefined function *max* that returns the maximum of its two parameters):

attributes to be used for the grammar - question (a) - already specified

<i>type</i>	<i>name</i>	<i>domain</i>	<i>nonterm.</i>	<i>meaning</i>
left	<i>a</i>	integer	<i>E</i>	number of summands at the same level of an expression
left	<i>amax</i>	integer	<i>E, S</i>	maximum number of summands at the same level of a subexpression

#	<i>syntax</i>	<i>semantics</i> - question (a)
1:	$S_0 \rightarrow E_1$	$amax_0 = \max(a_1, amax_1)$
2:	$E_0 \rightarrow E_1 \text{ ' + ' } E_2$	$a_0 = a_1 + a_2$ $amax_0 = \max(amax_1, amax_2)$
3:	$E_0 \rightarrow \text{ ' (' } E_1 \text{ ') ' }$	$a_0 = 1$ $amax_0 = \max(a_1, amax_1)$
4:	$E_0 \rightarrow n$	$a_0 = 1$ $amax_0 = 0$

Of course, there may be other solutions.

- (b) Being purely synthesized, the attribute grammar is one-sweep and also *L*.

(c) Here is the variant attribute grammar:

attributes to be added for question (c)

<i>type</i>	<i>name</i>	<i>domain</i>	<i>nonterm.</i>	<i>meaning</i>
right	p	integer	E	depth of an expression
left	$pmax$	integer	E, S	depth of the subexpression that has the maximum number of summands

#	<i>syntax</i>	<i>semantics</i> - question (c)
1:	$S_0 \rightarrow E_1$	$p_1 = 0$ if $a_1 > amax_1$ then $pmax_0 = 0$ else $pmax_0 = pmax_1$ endif
2:	$E_0 \rightarrow E_1 \text{ ' + ' } E_2$	$p_1 = p_0$ $p_2 = p_0$ if $amax_1 > amax_2$ then $pmax_0 = pmax_1$ else $pmax_0 = pmax_2$ endif
3:	$E_0 \rightarrow \text{' (' } E_1 \text{ ') '}$	$p_1 = p_0 + 1$ if $a_1 > amax_1$ then $pmax_0 = p_0 + 1$ else $pmax_0 = pmax_1$ endif
4:	$E_0 \rightarrow n$	$pmax_0 = -1$

Of course, there may be other solutions. The new attribute grammar is also one-sweep and L , since there is no dependency among attributes of brother nodes.